Algorithmique Fondamentale : Recherche Séquentielle et Dichotomique dans les Tableaux

Thème principal : Algorithmes de recherche dans les structures linéaires et complexité algorithmique

Synthèse exécutive : Ce cours aborde les méthodes fondamentales pour localiser un élément au sein d'un tableau. L'enjeu central repose sur la relation directe entre l'organisation des données (tableau trié ou non) et la performance temporelle de l'algorithme choisi. La recherche séquentielle parcourt le tableau de manière linéaire en O(n) sans aucune précondition sur les données. À l'inverse, la recherche dichotomique s'appuie sur le paradigme 'diviser pour régner' pour réduire l'espace de recherche de moitié à chaque étape, garantissant une complexité logarithmique en O(log n). Ce cours enseigne la maîtrise des deux algorithmes, l'analyse de leur complexité au pire des cas, la manipulation exacte des pointeurs d'indices (début, fin, milieu) et les critères d'arbitrage selon le contexte d'utilisation.

1. La Recherche Séquentielle (ou Linéaire)

La recherche séquentielle est la stratégie de recherche la plus intuitive et universelle. Elle consiste à passer en revue chaque élément du tableau successif jusqu'à trouver la cible ou atteindre la fin du tableau. Si le tableau contient n éléments, le pire des cas se produit lorsque la valeur cherchée se trouve à la toute dernière position ou n'existe pas dans le tableau : l'algorithme effectue alors exactement n comparaisons. Sa complexité est donc linéaire O(n). Son principal avantage est son universalité : elle s'applique à n'importe quelle structure séquentielle sans nécessiter un tri préalable coûteux.

- Parcours exhaustif du tableau indice par indice, du début (0) jusqu'à la fin (n-1).
- Aucune précondition requise : fonctionne sur des tableaux triés ou non triés.
- Complexité temporelle au pire des cas : O(n), où n est le nombre d'éléments.
- Arrêt anticipé dès que la valeur recherchée est trouvée (succès) ou parcours complet en cas d'absence (échec).

Cas concret enseignant : *Le professeur utilise l'analogie de la recherche d'une clé tombée dans un tas de feuilles mortes. N'ayant aucun ordre ni rangement possible dans un tas de feuilles, il n'existe aucune astuce : la seule solution consiste à soulever chaque feuille une par une jusqu'à trouver la clé.*

2. La Recherche Dichotomique (Diviser pour Régner)

La recherche dichotomique exploite l'ordre des éléments dans un tableau trié pour accélérer la recherche. À chaque étape, l'algorithme calcule l'indice du milieu : milieu = (debut + fin) // 2, puis compare la valeur T[milieu] à la cible recherchée. Si T[milieu] == cible, l'élément est trouvé. Si la cible est strictement inférieure à T[milieu], l'élément ne peut se trouver que dans la première moitié : le pointeur 'fin' devint 'milieu - 1'. Si elle est supérieure, on déplace 'debut' à 'milieu + 1'. En éliminant la moitié du tableau à chaque itération, l'espace de recherche décroît de façon exponentielle, permettant de trouver un élément parmi un million en environ 20 comparaisons seulement.

- Précondition stricte et obligatoire : le tableau doit être préalablement trié.
- Principe 'Diviser pour Régner' : élimination de 50 % des candidats restants à chaque comparaison.
- Gestion rigoureuse de trois pointeurs d'indices : debut, fin et milieu = (debut + fin) // 2.
- Complexité temporelle au pire des cas : O(log2 n) ou O(log n).

Cas concret enseignant : *Le professeur illustre cela avec le jeu du 'Devine un nombre entre 1 et 100'. Proposer 1, 2, 3... est une approche séquentielle inefficace. Proposer 50 permet d'éliminer d'un coup 50 nombres si la réponse est 'c'est plus grand'. On trouve la solution en 7 étapes maximum (log2(100) ≈ 6,64).*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne pas toujours utiliser la recherche dichotomique si elle est tellement plus rapide que la recherche séquentielle ?
Réponse du professeur : Parce que la recherche dichotomique exige impérativement un tableau trié. Si le tableau n'est pas trié, le coût pour le trier (au mieux O(n log n) avec un tri efficace) est supérieur au coût d'une simple recherche séquentielle O(n). La dichotomie n'est rentable que si le tableau est déjà trié ou si l'on effectue un très grand nombre de recherches sur un tableau stable.

Question étudiant : Comment calcule-t-on l'indice du milieu si le sous-tableau contient un nombre pair d'éléments ?
Réponse du professeur : On utilise la division entière (par exemple l'opérateur // en Python ou la troncature entière dans les autres langages). Pour debut = 0 et fin = 9 (10 éléments), milieu = (0 + 9) // 2 = 4. L'algorithme prend l'élément d'indice 4, ce qui divise le tableau en deux parties (0 à 3 et 5 à 9) sans poser aucun problème de logique.

■ Exercice & Démonstration d'Application

Énoncé : Soit le tableau trié T = [3, 8, 12, 15, 19, 24, 31, 42, 50, 61] comportant n = 10 éléments (indices 0 à 9). Appliquer pas à pas l'algorithme de recherche dichotomique pour rechercher la valeur 42.

Résolution pas à pas :
Étape 0 (Initialisation) : debut = 0, fin = 9. Étape 1 : Calcul du milieu -> milieu = (0 + 9) // 2 = 4. Valeur T[4] = 19. Comparaison : 42 > 19. La cible est plus grande, la recherche continue dans la partie droite. Mise à jour du pointeur debut : debut = milieu + 1 = 5. Étape 2 : Zone de recherche : debut = 5, fin = 9. Calcul du milieu -> milieu = (5 + 9) // 2 = 7. Valeur T[7] = 42. Comparaison : T[7] == 42. Succès !

Résultat : La valeur 42 est localisée à l'indice 7 en seulement 2 comparaisons (contre 8 comparaisons avec une recherche séquentielle).

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la condition impérative indispensable pour pouvoir appliquer une recherche dichotomique ?

R : Le tableau doit être strictement trié au préalable.

Q : Quelle est la complexité temporelle au pire des cas d'une recherche séquentielle sur un tableau de taille n ?

R : O(n) - Complexité linéaire.

Q : Combien de comparaisons au maximum la recherche dichotomique nécessite-t-elle pour un tableau trié de 1024 éléments ?

R : 10 comparaisons au maximum, car 2^10 = 1024 (log2(1024) = 10).

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765